IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 10 October 2025, accepted 11 November 2025, date of publication 14 November 2025, date of current version 2 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3633152

STANDARDS

Pre-Shared Key Authentication With EDHOC: The Security–Performance Tradeoff

ELSA LOPEZ PEREZ[1], (Student Member, IEEE), THOMAS WATTEYNE[1], (Senior Member, IEEE), RAFAEL MARIN-LOPEZ[2], CRISTINA ONETE[3], CLÉMENT PAPON[3], AND MALIŠA VUČINIĆ[1]

[1]Inria, 75013 Paris, France
[2]University of Murcia, 30003 Murcia, Spain
[3]CNRS 7252, University of Limoges, 87032 Limoges, France

Corresponding author: Elsa Lopez Perez (elsa.lopez-perez@inria.fr)

This work was supported in part by French National Research Agency under the France 2030 label under Grant NF-HiSec ANR-22-PEFT-0009, and in part by the Horizon Europe OpenSwarm Project under Grant 101093046.

ABSTRACT The rapid growth of the Internet of Things ecosystem has intensified the need for secure, resource-efficient communication protocols. The EDHOC protocol is a lightweight authenticated key-exchange protocol, recently developed by the Internet Engineering Task Force. EDHOC addresses the challenges of transport over constrained radio technologies and execution on constrained microcontroller units. In its standardized version, the key-exchange can be authenticated using signatures or static Diffie-Hellman keys. However, many Internet of Things deployments in the wild rely on Pre-Shared Keys. As such, the potential use of EDHOC in those deployments requires a new authentication method for this protocol, based on Pre-Shared Keys. Two variants of Pre-Shared Keys authentication in EDHOC are currently under consideration in the Internet Engineering Task Force LAKE working group. This paper presents a comprehensive analysis of these variants, examining their performance metrics, implementation complexity, and security and privacy considerations. Our evaluation focuses on computational time, memory usage, and deployment challenges in diverse Internet of Things ecosystems. Based on our analysis, we have formulated a recommendation to the Working Group, which has opted to adopt and standardize PSK2.

INDEX TERMS Internet of Things, communication security, authenticated key exchange, pre-shared keys, EDHOC.

NOMENCLATURE

3GPP	3rd Generation Partnership Project.
5G-AKA	5G Authentication and Key Agreement.
AEAD	Authenticated Encryption with Associated Data.
AKE	Authenticated Key-Exchange.
CBOR	Concise Binary Object Representation.
CC310	Crypto Cell 310.
COSE	CBOR Object Signing and Encryption.
DH	Diffie-Hellman.
DHE	Diffie-Hellman Exchange.
DTLS	Datagram Transport Layer Security.
DUT	Device Under Test.
EAD	External Authorization Data.
EDHOC	Ephemeral Diffie-Hellman Over COSE.
IETF	Internet Engineering Task Force.
IKM	Input Keying Material.
IoT	Internet of Things.
KCI	Key Compromise Impersonation.
LAKE	Lightweight Authenticated Key Exchange.
MAC	Message Authentication Code.

The associate editor coordinating the review of this manuscript and approving it for publication was Razi Iqbal.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

MitM	Meddler-in-the-Middle.
PFS	Perfect Forward Secrecy.
PRK	Pseudorandom Key.
PSK	Pre-Shared Key.
TLS	Transport Layer Security.

I. INTRODUCTION

The Internet of Things (IoT) has experienced a significant growth in the last years, with billions of devices being interconnected across various sectors. These devices form constrained environments, characterized by limited computational power, memory, and energy resources. As the IoT ecosystem expands, secure communication becomes a critical prerequisite to protecting sensitive data and preventing unauthorized access [1], [2], [3]. In response to these challenges, the Internet Engineering Task Force (IETF) has developed protocols for constrained environments.

A key development is Ephemeral Diffie-Hellman Over COSE (EDHOC) , an Authenticated Key-Exchange (AKE) protocol specifically tailored for constrained IoT networks. Developed by the IETF's Lightweight Authenticated Key Exchange (LAKE) working group, it utilizes Concise Binary Object Representation (CBOR) encoding and CBOR Object Signing and Encryption (COSE) object security to minimize message sizes and code footprint, making it well-suited for IoT applications. This approach enables efficient and secure communication, even in environments with limited resources.

The key features of the protocol include compact handshakes of around 100 bytes in size, a minimal requirement of three mandatory flights, independence from specific transport protocols, and efficient code size.

A. MOTIVATION

To further increase compatibility with today's diverse IoT environments, the IETF LAKE working group has lately focused on expanding the current authentication mechanisms within EDHOC to include Pre-Shared Key (PSK) based authentication. While authentication based on public-key cryptography enables secure-channel establishment between parties which have had no prior interaction, PSK-based authentication reduces complexity and increases flexibility. Indeed, many existing IoT schemes rely on PSK methods for authentication [4], [5]. Popular classes of IoT devices that commonly rely on PSK-based security mechanisms include smart home hubs or wearable devices, and they often implement protocols such as DTLS with PSK modes to ensure lightweight secure communication [6].

Moreover, an important strength of PSK-based authentication lies in its symmetric nature, which is inherently quantum-resistant. While the use of such primitives alone is insufficient to achieve post-quantum security (since PSK keys might be yielded by a quantum-vulnerable session and since hash functions behave differently in the QROM than in the ROM), using PSK-based authentication represents a good first step towards hybridization and potential quantum resistance for EDHOC.

The prevalence of PSK-based authentication is a main reason for its support – alongside with signature-based authentication – in recent IETF standards. This dual approach provides a crucial migration path, allowing organizations to gradually upgrade their IoT infrastructures while maintaining interoperability with existing PSK-based devices. As such, the integration of PSK authentication methods into EDHOC is a critical focus area within the IETF LAKE working group (also described in its charter[1]). Two protocol variants for PSK-based authentication in EDHOC were proposed in [7]. Based on the results presented in this paper, the Working Group has adopted PSK2 for standardization.

B. CONTRIBUTIONS

The contributions of this paper are three-fold:

- An in-depth evaluation of the two protocol variants proposed for standardization.
- A clear recommendation to the working group, which led to the adoption of the PSK2 variant for standardization.
- An open-source implementation of both protocol variants complementing an existing EDHOC implementation (without PSK-based authentication) in Rust, following draft version -01 [7].[2]

C. OUTLINE

The remainder of this article is organized as follows. Section II outlines the EDHOC protocol, including its cryptographic core and the protocol outline. Section III introduces the two variants of EDHOC with PSK-based authentication proposed for standardization. Section IV describes the followed methodology, including the methods used to evaluate and the hardware needed. Section V presents the evaluation results and Section VI presents the security considerations. Section VII and Section VIII discuss the results and provide recommendations to the working group, as well as proposing future research directions. Section IX surveys the state-of-the-art on EDHOC and other protocols used for authenticated key exchange in constrained environments. Finally, Section X concludes this article.

II. THE EDHOC PROTOCOL

EDHOC is an AKE protocol designed to be used by peers running on constrained devices over low-power IoT radio communication technologies. The keys derived in the protocol can later be used by applications such as OSCORE to protect data through authenticated encryption.

A. CRYPTOGRAPHIC CORE OF EDHOC

The cryptographic foundation of EDHOC is built on the MAC-then-Sign variant of Krawczyk's SIGMA-I protocol [8].

[1] https://datatracker.ietf.org/doc/charter-ietf-lake/
[2] https://github.com/ElsaLopez133/lakers/tree/integrated

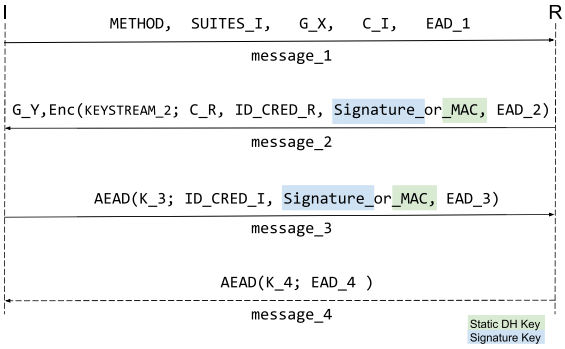


FIGURE 1. **The EDHOC message flow.** `G_X` **and** `G_Y` **represent the public ephemeral keys of the Initiator and the Responder, respectively. Field** `ID_CRED_I` **(resp.** `ID_CRED_R`**) denotes the authentication credentials of the Initiator (resp. Responder).** `METHOD` **is an integer (0-1-2-3) denoting the authentication method (see Table 1).** `SUITES_I` **is an ordered set of preferred algorithms. If** `METHOD` **is either 0 or 1 for the Initiator (resp. 0 or 2 for the Responder), then** `Signature_or_MAC` **equals Signature.**

SIGMA protocols form a family of key-exchange protocols that combine digital signatures and Message Authentication Code (MAC) functions to securely create authenticated Diffie-Hellman (DH) protocols. They provide mutual authentication, secure channels and a degree of privacy for the two peers (called *identity protection* in [8]).

In the SIGMA-I variant, the peer initiating the session, named "Initiator", receives identity protection against active attackers. The peer engaging in an already initialized session, called "Responder", is protected against passive attackers.

In order to authenticate the two protocol participants, EDHOC associates parties with public credentials (`CRED_I` for the Initiator and `CRED_R` for the Responder). These credentials can include signature public keys or static DH public keys. Authentication can be achieved through two essential means for each peer, as depicted in Table 1: signatures or a static DH key. The parties indicate their preferred means of authentication in the first message in the "authentication method" field. EDHOC optimizes message size by using MACs (8 B) instead of signatures (64 B in the most compact cipher suite) when static-DH is used for peer-authentication.

EDHOC includes a cipher suite (an ordered list of algorithms like authenticated encryption, hash function, and digital signature) in its first message to enable cryptographic agility.

B. PROTOCOL OUTLINE

The protocol consists of three mandatory messages, an optional fourth message, and an error message. Fig. 1 depicts the EDHOC message flow. Refer to Table 1 for the authentication methods (0-3) for each handshake.

Message 1. The Initiator sends the first message containing the setup information. This includes the authentication method, the cipher suite, the ephemeral Diffie-Hellman public key `G_X`, a connection identifier `C_I` (with no cryptographic purpose) and the additional information called external authorization data `EAD_1`.

TABLE 1. **Current authentication methods registered by IANA (0-1-2-3), and the new PSK authentication method (4).**

ID	Initiator	Responder
0	Signature	Signature
1	Signature	Static Diffie-Hellman
2	Static Diffie-Hellman	Signature
3	Static Diffie-Hellman	Static Diffie-Hellman
4	Pre-Shared Key	Pre-Shared Key

Message 2. If the Responder supports the method and cipher suites, it generates an ephemeral key pair (`Y`, `G_Y`) and computes the Diffie-Hellman shared secret `G_XY`. If the Responder's authentication method is static DH, the latter computes the ephemeral-static Diffie-Hellman shared secret `G_RX` from the key pair (`R`, `G_R`) corresponding to the static DH values. The Responder derives two intermediate pseudo-random keys using the Key Derivation Function `EDHOC_KDF`: `PRK_2e` and `PRK_3e2m`. The index in the key name indicates in what message and operation the key is involved. The Responder also computes a transcript hash `TH_2` over the first message and the ephemeral DH public key `G_Y`. When the Responder uses a Static Diffie-Hellman key, authentication is provided by a value `MAC_2`, instead of a signature. This MAC is derived by using `EDHOC_KDF` with `PRK_3e2m` and some additional information (`context_2`) that includes the transcript hash `TH_2` and its own credentials `CRED_R`. Otherwise (methods 0 and 2, see Table 1), the Responder authenticates via a signature. The parameters used in the signature algorithm are: *(1)* `ID_CRED_R`, an identifier to facilitate retrieval of `CRED_R`. *(2)* externally supplied data, `external_aad`, that includes `TH_2`, `CRED_R` and `EAD_2`, and *(3)* `MAC_2`. Message 2 is finally encrypted using key `KEYSTREAM_2`, derived from `PRK_2e`.

Message 3. The Initiator computes the shared secret `G_XY`, the decryption key `KEYSTREAM_2` derived from `PRK_2e`, and the pseudo-random keys `PRK_3e2m`, all used to verify message 2. If the Initiator authenticates via static DH with key pair (`I`, `G_I`), then it also computes the ephemeral-static Diffie-Hellman shared secret `G_IY`, needed to derive `PRK_4e3m`. Then, `I` verifies `R`'s credentials (using the MAC or Signature received in Message 2). If they verify, `I` computes a new transcript hash `TH_3` over `TH_2` and `R`'s credentials, and a MAC, `MAC_3`, using `EDHOC_KDF` on `PRK_4e3m` and some additional data (`context_3`) that includes `TH_3` and its own credentials `CRED_I`. Depending on the method used, the MAC is either sent raw or protected by a signature. It is further encrypted using keying material `K_3` and the initialization vector `IV_3`, if needed by the `AEAD` algorithm, derived from `PRK_3e2m`. The Authenticated Encryption with Associated Data (`AEAD`) allows for symmetric, randomized data encryption while providing secrecy and authenticity, i.e. nothing is known

about the plaintext and it is impossible to change the encrypted plaintext underlying the ciphertext without being detected.

Message 4. The Responder receives the third message and decrypts it to authenticate the Initiator. R can then send an optional fourth message containing external authorization data EAD_4 as acknowledgment, encrypted using the keying material K_4 / IV_4 derived from PRK_4e3m. Message 4 then serves as key confirmation.

C. SUPPORTING FUNCTIONS

Two additional functions are fundamental to the key-derivation: *(1)* EDHOC_Extract() that generates a fixed-length pseudorandom key (PRK) from a salt and an Input Keying Material (IKM) , and *(2)* EDHOC_Expand() that takes the previously generated PRK, a sequence named "info", and the key length, and outputs several additional pseudo-random keys that are then used both in the EDHOC_KDF and as output keying material. The definitions of both functions depend on the hash algorithm of the cipher suites. These functions are typically instantiated using HKDF, described in RFC5689 [9].

III. EDHOC PSK-BASED AUTHENTICATION UNDER STANDARDIZATION

The IETF LAKE working group is currently standardizing an authentication method for EDHOC based on PSKs. The two proposed protocol variants, PSK1 and PSK2, are detailed in [7]. Their essential differences lie in: the key derivation schedule (see Figs. 3 and 5), the message in which the PSK identifier (see the explanation below) is sent, and the number of mandatory protocol messages.

PSK-based authentication in EDHOC can be used for fresh sessions, where the PSK is distributed using an out-of-band channel, or resumption sessions, where the PSK is derived from previous handshakes using any of the supported authentication methods (0-1-2-3). In both cases, the PSK is used in combination with the ephemeral Diffie-Hellman shared secret to guarantee forward secrecy.

Since each device might contain several PSKs, each PSK is associated with a PSK identifier, sent in Message 1 in the first protocol variant, and in Message 3 in the second. This identifier acts as the identity of both the Initiator and the Responder.

In this section, we detail the message flow of each variant proposed for standardization, highlighting the differences with respect to the EDHOC protocol described in Section II.

A. PSK1

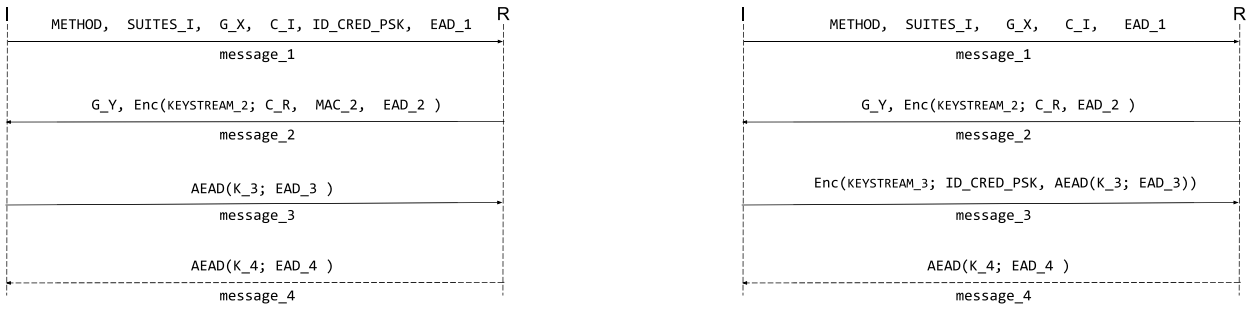
PSK1 closely follows EDHOC's original message flow, just adding the PSK identifier in Message 1 (see Fig. 2).

- **Message 1:** Similar to message 1 in EDHOC, but including a PSK identifier ID_CRED_PSK, sent in the clear.
- **Message 2:** As for Message 2 described in Section II-B, if the method and cipher suites are supported by the Responder, it generates an ephemeral key pair (Y, G_Y) and computes the Diffie-Hellman shared secret G_XY. The Responder computes PRK_2e, PRK_3e2m, which is derived using CRED_PSK, and a transcript hash TH_2 over the first message and the ephemeral DH public key G_Y. It then computes MAC_2 using EDHOC_KDF with PRK_3e2m and additional information (context_2) containing the transcript hash TH_2 and the PSK credential CRED_PSK. Message 2 is encrypted using key KEYSTREAM_2, derived from PRK_2e.
- **Message 3:** The Initiator computes the shared secret G_XY, the decryption key KEYSTREAM_2 derived from PRK_2e, and the pseudo-random key PRK_3e2m. Then, I computes a new transcript hash TH_3 including the PSK credential, CRED_PSK. Contrary to EDHOC (Fig. 1), no MAC_3 is computed. Authentication is guaranteed by means of the AEAD, with TH_3 as external_aad.
- **Message 4.** This message is mandatory if explicit key-confirmation is required, but remains optional if implicit key-confirmation is acceptable (assuming an explicit key-confirmation follows at the application later). The message can contain external authorized data EAD_4 as acknowledgment, encrypted using key K_4 and initialization vector IV_4, if needed by the AEAD algorithm, derived from PRK_4e3m.

B. PSK2

PSK2 deviates from PSK1 by sending the ID_CRED_PSK in Message 3, encrypted using a key derived from the ephemeral shared secret G_XY. In this case, the Initiator authenticates first (see Fig. III). The key derivation schedule is shown in Fig. 5

- **Message 1:** Message 1 remains as in EDHOC (see Section II-B).
- **Message 2:** As opposed to Message 2 in EDHOC and in variant PSK1, in PSK2 no MAC_2 calculation is needed. Thus, the Responder needs only to compute the Diffie-Hellman shared secret G_XY, PRK_2e, PRK_3e2m, and the transcript hash TH_2. Message 2 is encrypted using key KEYSTREAM_2, derived from PRK_2e.
- **Message 3:** The Initiator computes the shared secret G_XY, the decryption key KEYSTREAM_2 derived from PRK_2e, and the pseudo-random keys PRK_3e2m. The Initiator then computes a new transcript hash TH_3 including the PSK credential, CRED_PSK. The Initiator generates CIPHERTEXT_3, a bit string obtained XOR encrypting PLAINTEXT_3 = (ID_CRED_PSK, CIPHERTEXT_3B) with key KEYSTREAM_3, where CIPHERTEXT_3B is the output of the AEAD algorithm using EAD_3 as



(a) Message flow of EDHOC with symmetric keys. PSK1.

(b) Message flow of EDHOC with symmetric keys. PSK2.

FIGURE 2. Message flow of EDHOC with PSK. `METHOD` **is an integer (0-1-2-3-4) indicating the authentication method.** `SUITES_I` **is an ordered set of preferred algorithms.** `G_X` **and** `G_Y` **are the public ephemeral DH keys of the Initiator and the Responder, respectively.** `C_I` **and** `C_R` **are connection identifiers of Initiator and Responder, respectively.** `ID_CRED_PSK` **is the credential identifier of the PSK.**

plaintext and `TH_3`, `ID_CRED_PSK` and `CRED_PSK` as externally supplied data, `external_aad`.

- **Message 4.** To achieve mutual authentication, Message 4 is mandatory in PSK2. In particular, it is needed to authenticate the Responder. Therefore, this variant provides explicit key confirmation. Message 4 is encrypted using `PRK_4e3m`, which is derived from `CRED_PSK`.

IV. METHODOLOGY

This section details the experimental setup and procedures used to evaluate the protocol's performance, including message size, number of operations, energy and time consumption, and memory consumption. We describe the hardware and software tools used, as well as the specific techniques for measuring energy and memory consumption and execution time.

A. MEASUREMENT SETUP

We used the nRF52840 development kit as the Device Under Test (DUT) to evaluate the two PSK protocol variants, integrated into the Rust-based *lakers* EDHOC implementation (available at open-source at[3]). This system-on-chip features a 32-bit ARM Cortex-M4 processor running at 64 MHz and a BLE radio, which we used at the physical layer of the communication without relying on the upper layers of the BLE stack. Cryptographic operations were accelerated using the ARM Crypto Cell 310 (CC310) . The nRF52840 was chosen as a representative platform for resource-constrained IoT devices, as its low-power ARM Cortex-M4 processor reflects the capabilities of a wide range of modern sensors and connected devices. While our experiments are limited to this specific hardware, the results provide a strong foundation for assessing the protocol's performance in similar constrained environments. The EDHOC implementation used for this evaluation is a generic, open-source Rust library. While the current evaluation focuses on a specific hardware platform for controlled measurements, the underlying code is written to be hardware-agnostic. For all the measurements presented in Section V, we use the compact encoding for the PSK identifier specified in RFC9528 [10].

[3] https://github.com/ElsaLopez133/lakers/tree/integrated

B. PERFORMANCE METRICS AND PROCEDURES

1) MESSAGE SIZE

Message sizes were determined by parsing and printing the logs of the protocol exchanges.

2) NUMBER OF OPERATIONS

The number of cryptographic and protocol operations was determined by counting them directly within the code execution flow.

3) HANDSHAKE DURATION AND ENERGY CONSUMPTION

For precise measurement of execution time, we used the Saleae Logic 8 connected to designated GPIO pins on the nRF52840 development board, which are toggled at the beginning and end of relevant code sections. The Otii Arc supplies power to the DUT and simultaneously measures current and voltage to calculate energy consumption. To ensure timing correlation with the protocol execution, the Otii Arc's GPIO was connected to a common GPIO monitored by the Saleae Logic 8 logic analyzer, enabling synchronization between energy measurements and protocol execution phases.

4) MEMORY CONSUMPTION

To measure flash usage, we analyzed `.map` files (excluding the crypto backend), calculating it as the sum of `.text`, `.rodata`, and `.data` sections. RAM usage was determined by summing the `.data` and `.bss` sections.

Stack usage was measured using a memory painting technique: *(1)* painting the stack with a magic number, *(2)* running the program to overwrite parts of the stack, and *(3)* using a script with `probe-rs` to analyze the remaining painted areas to calculate stack usage.

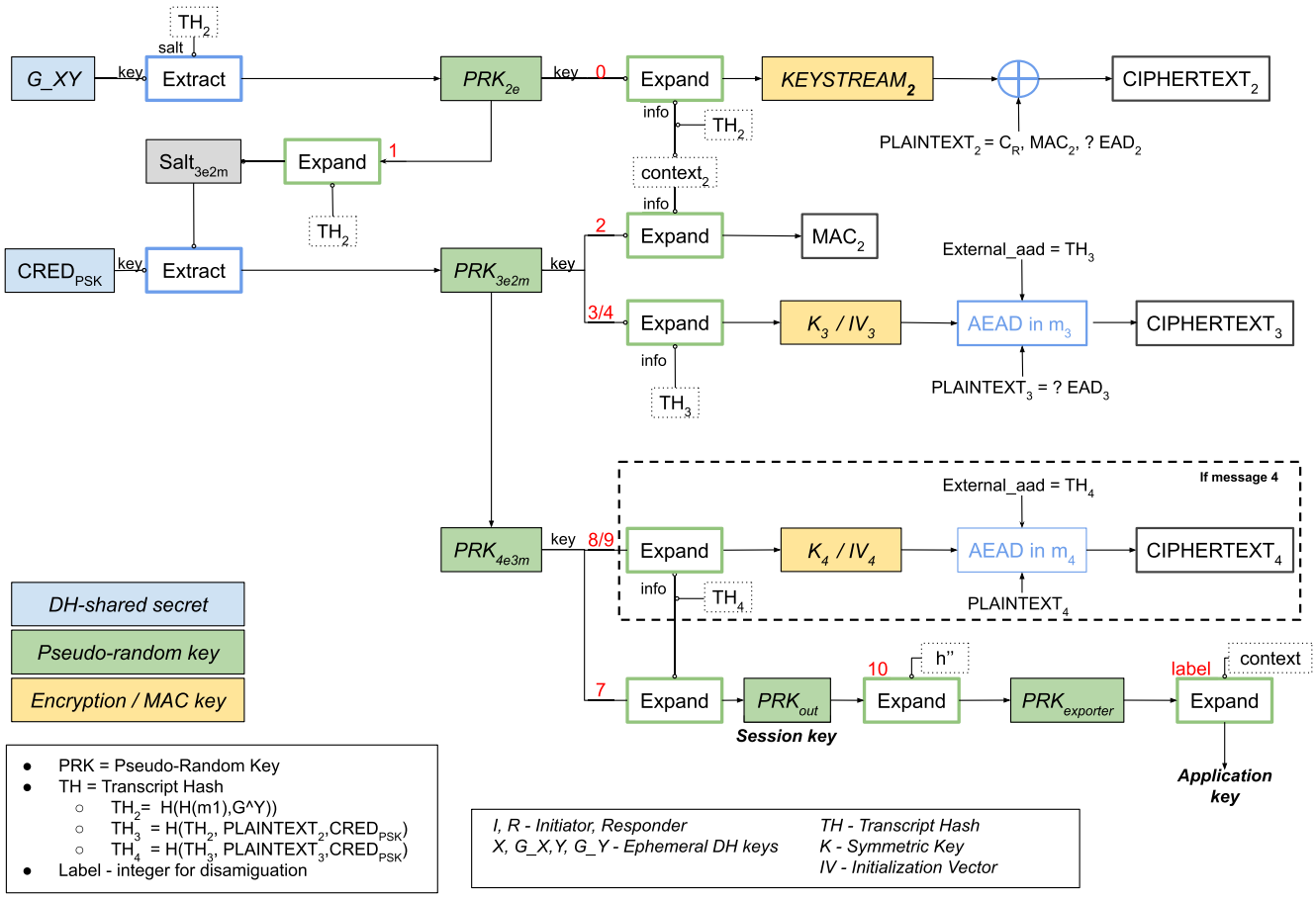


FIGURE 3. Key derivation scheme for PSK1.

V. PERFORMANCE EVALUATION

This section presents the results of our performance evaluation, building upon the methodology described in the previous section.

A. MESSAGE SIZE

Table 2 compares message sizes in bytes. PSK1's **Message_1** is one byte larger due to the unencrypted PSK identifier `ID_CRED_PSK`. **Message_2** contains 9 extra bytes in the case of PSK1. This is because this variant includes the encryption of `MAC_2`, 8 bytes long, with an additional byte for length indication. **Message_3** is two bytes longer in the case of PSK2, due to the addition of the encryption of the PSK identifier (one byte for the PSK identifier and an extra byte for CBOR encoding of the ciphertext). Finally, PSK1 omits **Message_4** whereas PSK2's Message_4, containing the `AEAD` of an empty `EAD_4`, is 9 bytes (`AEAD` tag).

B. NUMBER OF OPERATIONS

Table 3 details cryptographic operations. Both PSK1 and PSK2 involve two asymmetric (DH) operations. PSK1 uses three symmetric operations (encryption and MACs), while PSK2 uses four due to an extra encryption in Message 3 and the `AEAD` in Message 4. There are the same number of hash operations (`TH_X`), hence, they are not accounted for.

TABLE 2. Comparison of EDHOC message sizes (in bytes) for the two protocol variants. m_x denotes the message number (1,2,3,4).

Method	m_1	m_2	m_3	m_4	Total
PSK1	38	44	9	-	91
PSK2	37	35	11	9	92

TABLE 3. Comparison of cryptographic operations in PSK1 and PSK2. "Asym." stands for asymmetric. "Sym." stands for symmetric.

Method	Operations per Message				Total
	m_1	m_2	m_3	m_4	
PSK1	1 Asym (DH)	1 Asym 2 Sym	1 Sym	-	2 Asym 3 Sym
PSK2	1 Asym (DH)	1 Asym 1 Sym	2 Sym	1 Sym	2 Asym 4 Sym

C. HANDSHAKE DURATION AND ENERGY CONSUMPTION

Fig. 7 and Table 4 present runtime and energy consumption data obtained using an Otii Arc power profiler over 500 handshakes. Both PSK methods are faster than the Static

```
PRK_2e         = EDHOC_Extract( TH_2, G_XY )
PRK_3e2m       = EDHOC_Extract( salt_3e2m, CRED_PSK )
PRK_4e3m       = PRK_3e2m
SALT_3e2m      = EDHHOC_KDF( PRK_2e, 1, TH_2, hash_length )
SALT_4e3m      = EDHOC_KDF( PRK_3e2m, 5, TH_3, hash_length )
KEYSTREAM_2    = EDHOC_KDF( PRK_2e, 0, TH_2, plaintext_length )
MAC_2          = EDHOC_KDF( PRK_3e2m, 2, context_2, mac_length_2 )
K_3            = EDHOC_KDF( PRK_4e3m, 3, TH_3, key_length )
IV_3           = EDHOC_KDF( PRK_4e3m, 4, TH_3, iv_length )
K_4            = EDHOC_KDF( PRK_4e3m, 8, TH_4, key_length )
IV_4           = EDHOC_KDF( PRK_4e3m, 9, TH_4, iv_length )
PRK_out        = EDHOC_KDF( PRK_4e3m, 7, TH_4, hash_length )
PRK_exporter   = EDHOC_KDF( PRK_out, 10, h'', hash_length )
context_2      = << C_R, ID_CRED_PSK, TH_2, CRED_PSK, ? EAD_2 >>
```

FIGURE 4. Pseudocode for the key derivation of PSK1.

TABLE 4. Comparison of current consumption and time duration for an example of a complete handshake for different EDHOC authentication methods.

Role	Method	Max Current (mA)	Min Current (mA)	Avg Current (mA)	Time (ms)
Init.	PSK1	15.16	10.96	13.83	91.34
	PSK2	16.53	12.26	15.20	93.60
	STAT	14.96	10.73	13.61	131.15

DH–Static DH (Stat-Stat) method. PSK1 is slightly faster than PSK2, due to PSK2's additional Message_4 and the use of encryption in Message_3. These values represent a best-case scenario where no signatures are required and message sizes are reduced. Note that we only present results of PSK variants and Stat-Stat method since methods using Signatures are not yet implemented.

D. MEMORY CONSUMPTION

Fig. 8 illustrates memory consumption. The results show similar initiator stack consumption across all methods. However, for the Responder, PSK1 uses the least stack memory, while STAT consumes the most. Regarding flash footprint, PSK1 exhibits the lowest usage for both peers, and STAT the highest. PSK2's larger flash footprint compared to PSK1 is attributed to the additional code for the extra encryption and processing in Message 3. The lower flash consumption of PSK1 compared to STAT is due to the removal of some `MAC` computations. Our findings differ from those of Fedrecheski et al. [11] due to our use of a pure Rust implementation with an Embassy async runtime instead of the RIOT operating system.

VI. SECURITY AND PRIVACY CONSIDERATIONS

The security requirements of EDHOC were defined by Vučinić et al. [12]. Security analyses are provided by Davis and Günther [13], Günther and Tshibumbu Mukendi [14], and Cottier and Pointcheval [15]. A full provable-security treatment for EDHOC with PSK1/2 is outside the scope of this paper. Instead, we provide a high-level description of the security model, including oracles, party and instance characterization, and the threat model – and analyze the comparative security and privacy benefits of PSK1/2.

We first recall the main properties that EDHOC sets out to achieve, as described in [12]:

- **Mutual authentication.** Both peers (freshly) authenticate each other in each handshake, even with long-term key compromise (Key Compromise Impersonation), resisting active Meddler-in-the-Middle (MitM) adversaries, replays and identity misbinding. Selfie attacks such as those described in [16] are out of scope.
- **Confidentiality.** Only the two peers running the handshake can learn the derived session secret. In addition, keys of past sessions remain secure even if the peers' long-term keys are compromised (forward-secrecy), achieving Forward-Secure AKE [15] for secure channels.
- **Cryptographic strength.** The target security level of the protocol's key must be of 128 bits.
- **Identity Protection.** EDHOC must protect the identity of one of the peers against active attackers and the identity of the other against passive attackers.
- **Protection of External Data.** The protocol should support integration of external security applications using External Authorization Data (EAD) message fields. Auxiliary data must be transmitted as securely as protocol data in each flight.
- **Downgrade Protection.** The protocol must support cryptographic agility, including modularity and negotiation of preferred cryptographic primitives.

A. OBTAINING AND USING PSKS

EDHOC is agnostic to how PSKs are obtained, but this impacts the security and privacy of the obtained protocol. We highlight four main considerations:

Use of PSKs. A PSK can be used a single time, multiple times (up to a pre-set number of uses), or unlimited-use (long-term). We assume that, every time a PSK `CRED_PSK` is used, it is associated with the same identifier `ID_CRED_PSK`.

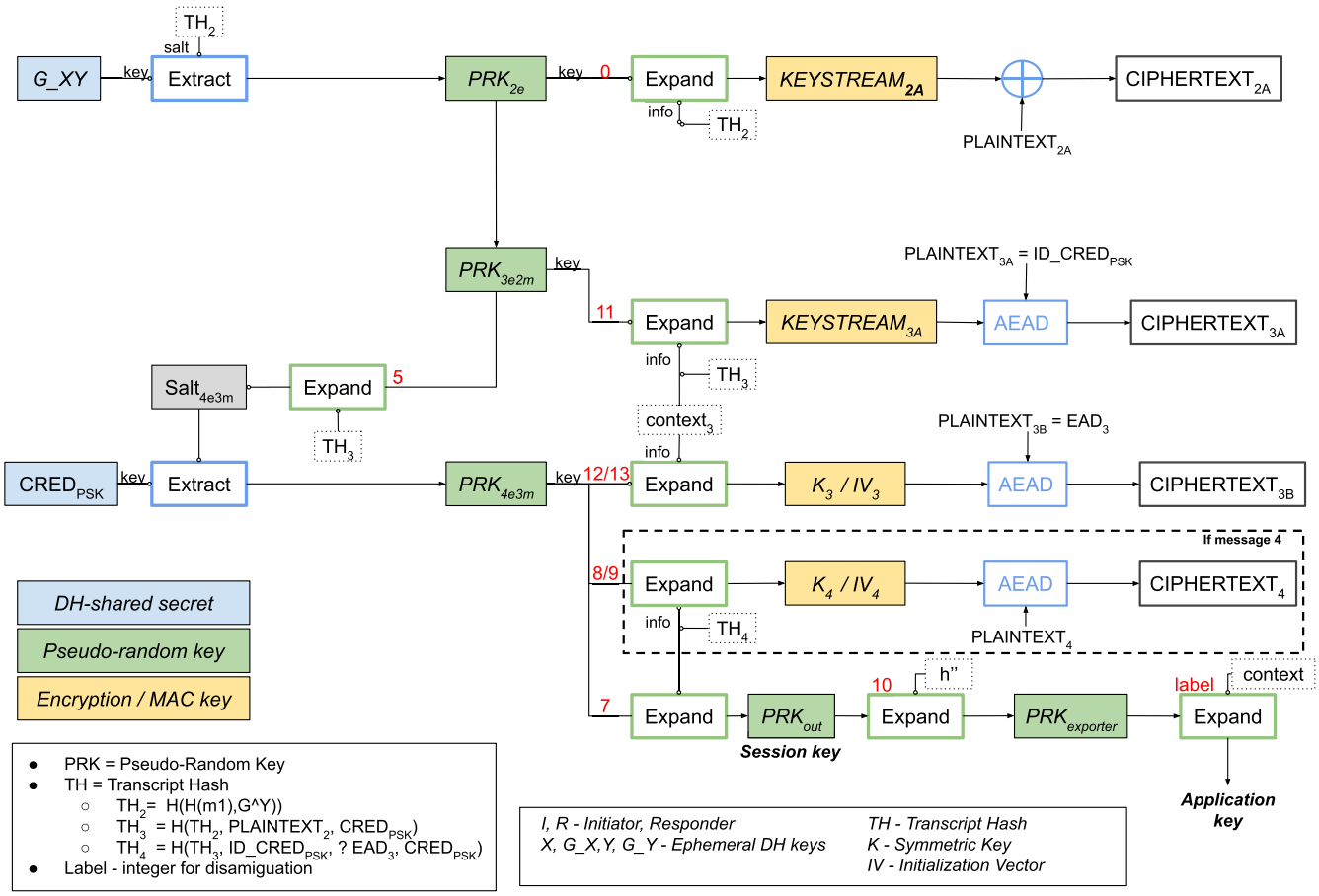


FIGURE 5. Key derivation scheme for PSK2.

Hardcode vs. compute. PSKs can be obtained in multiple ways. For instance, they could be hardcoded, in a symmetric fashion, on partnered devices. While this is impractical, it may be useful in use-cases such as logistics, where it is clear which devices are likely to be paired. A more realistic method could be to include PSK derivation keyed with `PRK_exporter` in the key-schedule, similarly to TLS 1.3. Brzuska et al. describe secure means of achieving security for export secrets [17]. Note that, while computing PSKs as part of the key-schedule ensures the freshness of PSKs, it induces a linkability between sessions which damages session-unlinkability [18].

To ticket or not to ticket. In settings in which one party, typically the server in client-server protocols, may be unwilling to store large quantities of PSK data, we also account for the potential use of session tickets [19]. Specifically, we assume that, while clients (initiators in our scenario) might compute a new PSK at the end of a protocol session and store it until it is used, servers symmetrically-encrypt PSK material in a potential session-ticket, which must later be used by the client in order to enable PSK-based authentication. These considerations give rise to eight separate cases for our analysis, depicted in Table 5.

TABLE 5. Types of PSKs.

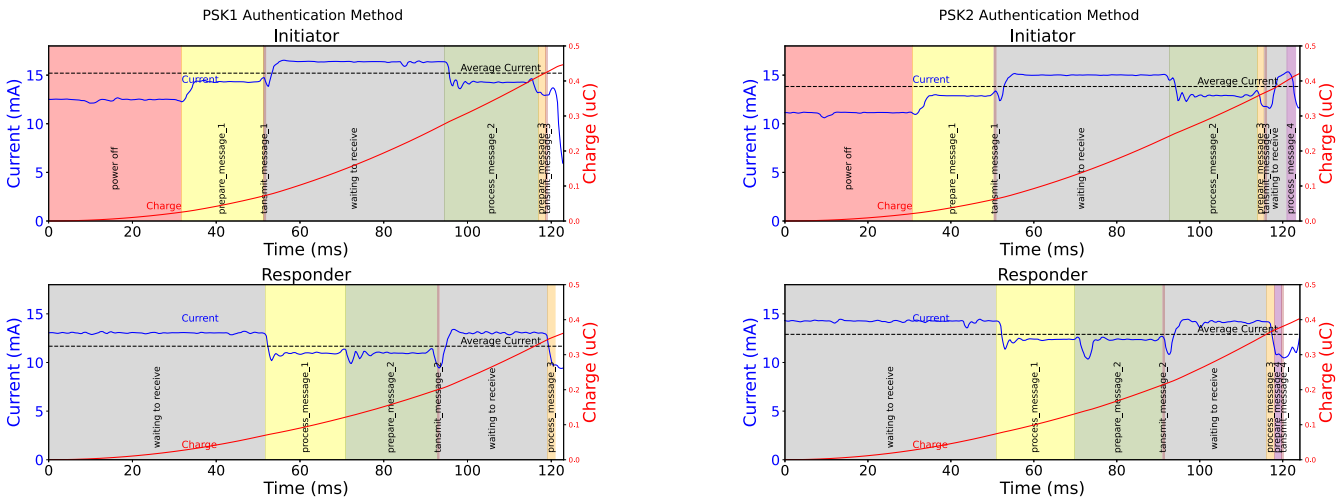
Single-use `CRED_PSK`			
Computed		Stored	
Ticket	No ticket	Ticket	No ticket
1	2	3	4

Multiple-use `CRED_PSK`			
Computed		Stored	
Ticket	No ticket	Ticket	No ticket
5	6	7	8

The nature of `ID_CRED_PSK`. In the context of our security and privacy analysis, it is imperative that the identifier `ID_CRED_PSK` has several properties: it must be unique per each PSK, it must provide no information either on the PSK value itself, on its owners, or on other state data. In particular, the identifier must also not help adversaries break EDHOC's security (e.g., link sessions, reveal secrets/confidential messages). A good practice is to choose identifiers as random values, chosen from a very large set, in order to prevent attacks.

```
PRK_2e        = EDHOC_Extract( TH_2, G_XY )
PRK_3e2m      = PRK_2e
PRK_4e3m      = EDHOC_Extract( salt_4e3m, CRED_PSK )
SALT_3e2m     = EDHHOC_KDF( PRK_2e, 1, TH_2, hash_length )
SALT_4e3m     = EDHOC_KDF( PRK_3e2m, 5, TH_3, hash_length )
KEYSTREAM_2   = EDHOC_KDF( PRK_2e, 0, TH_2, plaintext_length )
KEYSTREAM_3   = EDHOC_KDF( PRK_3e2m, 11, TH_3, key_length )
K_3           = EDHOC_KDF( PRK_4e3m, 12, TH_3, key_length )
IV_3          = EDHOC_KDF( PRK_4e3m, 13, TH_3, iv_length )
K_4           = EDHOC_KDF( PRK_4e3m, 8, TH_4, key_length )
IV_4          = EDHOC_KDF( PRK_4e3m, 9, TH_4, iv_length )
PRK_out       = EDHOC_KDF( PRK_4e3m, 7, TH_4, hash_length )
PRK_exporter  = EDHOC_KDF( PRK_out, 10, h'', hash_length )
```

FIGURE 6. Pseudocode for the key derivation of PSK2.



(a) Handshake duration and energy consumption using PSK1 authentication method.

(b) Handshake duration and energy consumption using PSK2 authentication method.

FIGURE 7. Handshake duration and energy consumption of PSK variants in EDHOC.

B. SECURITY MODEL IN THE PRESENCE OF PSKS

We consider the security of PSK-authenticated EDHOC in a standard game-based setting, in which security is formalized in terms of an *adversary*, playing a security game against a *challenger*, simulating honest parties. The adversary $\mathcal{A}$ is able to interact with an execution environment consisting of a polynomial number N of parties. Each party P may run multiple parallel protocol sessions. Each protocol session is run by an *instance* of party P, labelled as π_{P}^{q}, and an *instance* π_{Q}^{j} of its partner Q. Multiple instances of the same party P share that party's long-term parameters and state, but each instance computes and stores a number of session-specific values we call attributes, including: a unique session identifier $\pi_{\mathsf{P}}^{q}.\mathsf{sid}$ (consisting of the values $(\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$), the identity of that instance's partner $\pi_{\mathsf{P}}^{q}.\mathsf{PNID}$, the role P plays in that session (initiator or responder), and the ultimately-computed session key $\pi_{\mathsf{P}}^{q}.\mathtt{PRK_out}$.

At the core of our security model is the handling of PSKs. Each party P will need to keep track of a database $\mathsf{P}.\mathcal{DB}$ of PSKs, with entries of the form $(\mathtt{ID_CRED_PSK}, \mathtt{CRED_PSK}, \mathsf{ctn}, \mathsf{max}, \mathsf{ticket}, \mathsf{pskmeth})$, which specify the PSK identifier, the PSK itself, the number of times ctn the credential can still be used and respectively the maximal number max of uses (both these values can be set to ∞ if PSKs are unlimited use), the value of the session ticket encrypting the server state (if tickets are used), and a value $\mathsf{pskmeth}$, denoting the PSK type among the eight that we consider in Table 5.

While party P is associated with the full database of PSK values $\mathsf{P}.\mathcal{DB}$, each instance π_{P}^{q} of that party will employ a single credential included in $\mathsf{P}.\mathcal{DB}$, which we denote as $\pi_{\mathsf{P}}^{q}.\mathsf{CPSK_{in}}$ and call the *current credential*. If PSKs are computed at each session (rather than hardcoded), then the instance will also eventually output a new PSK

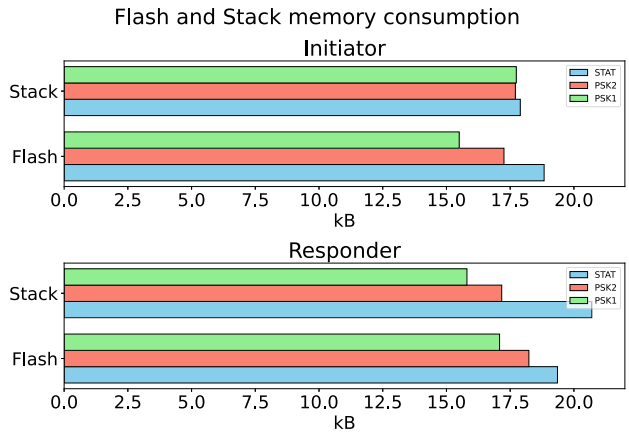


FIGURE 8. Flash and Peak Stack memory consumption for PSK authentication methods and Stat-Stat authentication method.

credential $\pi_{\mathsf{P}}^{q}.\mathsf{CPSK}_{\mathsf{out}}$, potentially with a corresponding ticket. Each instance leads to the update of the party's PSK database $\mathsf{P}.\mathcal{DB}$.

1) THREAT MODEL

The adversary interacts with the challenger (which simulates parties and instances) and has *oracle* access to the execution environment. Oracles allow the adversary to learn information about protocol messages and parties' private input – without learning the private values themselves. Each oracle query made by $\mathcal{A}$ being followed with a response by the challenger, as follows: `oRegister` (allows the adversary to register parties arbitrarily into the system – parties will generate one long-term credential: a secret key for ticket encryption), `oNewSecret` (allows the adversary to create honest or corrupted/malicious sessions and secrets), `oNewSession` (creates a new instance for a user P, which has a given role and intended party), `oSend` (which allows $\mathcal{A}$ to simulate the protocol run for created instances), `oCorrupt` (gives $\mathcal{A}$ the long term secret encryption key of a party), `oReveal` (reveals the session key `PRK_out` of a session between two endpoints), `oTicketReveal` (reveals the long-term secret key and the additional data used to encrypt a particular ticket), `oPSKReveal` (reveals the PSK used in a session between two endpoints), and $\texttt{oTest}_b$ (this oracle can only be queried once, and depending on the value of the bit b, inaccessible to $\mathcal{A}$, it returns either a random key or the real session key `PRK_out` of instance π_{P}^{q}). The adversary's access to such oracles allows us to consider arbitrary (rather than known and fixed) attack strategies and hence, arrive at a better security.

2) MUTUAL AUTHENTICATION/CONFIDENTIALITY

In order to prove that mutual authentication and confidentiality are achieved in this threat model, we define a security game in which the adversary ultimately aims to distinguish one bit of the key computed by two honest party instances within a session. The adversary $\mathcal{A}$ has access to all the previously described oracles. Its goal is to distinguish from random a secret bit b, drawn at random by the challenger, which is used by the $\texttt{oTest}_b$ oracle in the unique query allowed to the adversary. Its oracle access renders $\mathcal{A}$ more powerful than in reality – which is a sound approach, as it makes for stronger, more secure protocols, but which also allows $\mathcal{A}$ to abuse its superpower-access in order to win the game trivially. For instance, $\mathcal{A}$ could corrupt one party in order to learn their private keys, then impersonate that party and trivially compute the correct key (distinguishing all its bits). As a result, the security game rules out artificial attacks, which are either a product of the way the protocol works, or of the security model semantics. Adversaries cannot win against sessions that are not **sound**. Soundness ensures that only two instances can form a session with the same session identifier sid (we say that they are partnered instances), and that two partnered instances have opposite roles, use the same PSK credentials, and if run a protocol session until the end, they derive the same session key (in practice this is ensured by the protocol run). The adversary must also respect the **ExplicitAuth** requirement, guaranteeing that for any unrevealed instance (through `oPSKReveal` or `oTicketReveal`) that reaches the end of a protocol run, there exists a partnered instance that is not revealed and that also runs the protocol to the end. If the adversary does not respect these two previous conditions, the challenger returns $b' = 1$.

Finally to hope to win, the adversary must ensure that the tested instance is **fresh**, i.e., this instance must have computed a session key that is not revealed (through `oReveal`), that the PSK material of this instance has not been revealed (thanks to `oPSKReveal` or `oTicketReveal`) and that the associated endpoint has not been corrupted (through `oCorrupt`), but also that the partner instance (resulting from **Sound** and **ExplicitAuth**) verified the same condition regarding corruption and revelation. If $\mathcal{A}$ does not respect this requirement, the challenger returns a random bit b'. Note that, while the adversary has to *win* against a fresh instance, other unfresh instances might exist in the game. This renders our security statements stronger. This game in depicted in Figure 9 below.

3) IDENTITY PROTECTION

In PSK-mode (regardless of the protocol variant), a single PSK credential is bound to two entities (the endpoints) and thus the degree of protection becomes equivalent for both peers. The identity-protection definition of Cottier and Pointcheval [15] is stronger than the intuition of that notion indicated by Krawczyk [8]. In the [15] notion, an active Meddler-in-the-Middle (MitM) can impose the use of a specific `CRED_PSK` and then distinguish a legitimate owner of that PSK from a different user.

However, passive security might be achieved, according to the security game depicted in Figure 10 below. We allow an adversary to choose two pairs of initiators and responders,

$\mathbb{E}^{\mathsf{C-MA}}_{\mathsf{EDHOC\ PSK\ 1-2},\mathcal{A}}(\lambda)$

$\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda), \mathbf{Endpoints} \leftarrow \emptyset$

$b \leftarrow\$ \{0,1\}$

$$\mathcal{O}_{\mathsf{C-MA}} \leftarrow \left\{ \begin{array}{l} \mathtt{oRegister}(\cdot), \\ \mathtt{oNewSecret}(\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oNewSession}(\cdot,\cdot,\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oCorrupt}(\cdot), \mathtt{oReveal}(\cdot), \\ \mathtt{oTicketReveal}(\cdot), \\ \mathtt{oPSKReveal}(\cdot,\cdot), \\ \mathtt{oTest}_b(\cdot), \mathtt{oSend}(\cdot,\cdot) \end{array} \right\}$$

$(e, \Xi, b') \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{C-MA}}}(\lambda)$

if $\neg\mathsf{Sound}$ **then return** 1

if $\neg\mathsf{ExplicitAuth}$ **then return** 1

if $\neg\mathsf{Fresh}(\pi^e_\Xi)$ **then** $b' \leftarrow\$ \{0,1\}$

if $b = b'$ **then return** 1

else return 0

FIGURE 9. **The EDHOC PSK Confidentiality and Mutual Authentication security game. Here** `Setup` **is a general algorithm defining the parameters for the protocol run (e.g., the group that will be used) based on the security parameter** λ**.** Endpoints **denotes the set of endpoints that will be registered by** $\mathcal{A}$**.**

assumed to already have a shared PSK. Our challenger then selects one of the two pairs and faithfully simulates the protocol between these two endpoints. Upon receiving the messages, the adversary must guess which pair was chosen by the challenger in order to win the security game. Note here that $\mathcal{A}$ has no access to any oracle previously described. This security game thus highlights the importance of ensuring that `ID_CRED_PSK` does not reveal any information about the endpoints identities. It is also worth noting that refreshing the PSKs for each session can be a good practice to prevent session linkage, and thus prevent the adversary from learning information about identities.

C. COMPARATIVE ANALYSIS OF PSK1 AND PSK2

1) PSK1 ANALYSIS

Compare to EDHOC, PSK1 has minor changes, but our security model leads to weaker security guarantees. It can run in 3 messages (for implicit authentication) or 4 (for explicit). The main modifications made to the protocol are the following:

- **`ID_CRED_PSK` sent in clear.** Sending the identifier of the PSK credentials in the clear will harm the *identity privacy* of PSK1-authenticated EDHOC in the following cases:
  (1) If `ID_CRED_PSK` explicitly leaks linking information to other sessions;
  (2) If multiple-use PSKs allow passive attackers to link sessions. Active attackers can distinguish legitimate/illegitimate PSK possessors, even with single-use PSKs;
  (3) If PSKs are derived at the end of each session, parties that have corrupted the session-key material of prior sessions may learn `ID_CRED_PSK` and thus link sessions where it is used.

$\mathbb{E}^{\mathsf{Id-protection}-b}_{\mathsf{EDHOC\ PSK\ 1-2},\mathcal{A}}(\lambda)$

$(\mathtt{CRED_I}_0, \mathtt{CRED_R}_0) \leftarrow \mathcal{A}(\mathsf{Endpoints})$

$(\mathtt{CRED_I}_1, \mathtt{CRED_R}_1) \leftarrow \mathcal{A}(\mathsf{Endpoints})$

// *$\mathcal{A}$ choses two pairs of identity which have a previous common EDHOC session run leading to a PSK session*

$(\mathsf{ID_I}, \mathsf{ID_R}) \leftarrow (\mathtt{CRED_I}_b, \mathtt{CRED_R}_b)$

// *The challenger choses one of the two pairs according to the bit b of the experiement...*

$(\mathtt{ID_CRED_PSK}, \mathtt{CRED_PSK}) \leftarrow (\mathsf{ID_I}, \mathsf{ID_R})$

// *...and gets the associated PSK credentials*

$\mathtt{message_1} \leftarrow \mathtt{InitRun1}(\mathsf{ID_I}, \mathtt{ID_CRED_PSK})$

$\mathtt{message_2} \leftarrow \mathtt{RespRun1}(\mathsf{ID_R}, \mathtt{message_1})$

$\mathtt{message_3} \leftarrow \mathtt{InitRun2}(\mathsf{ID_I}, \mathtt{message_2})$

$\mathtt{message_4} \leftarrow \mathtt{RespRun2}(\mathsf{ID_R}, \mathtt{message_3})$

// *The challenger simulates the protocol run*

$b' \leftarrow \mathcal{A}(\mathtt{message_1}, \mathtt{message_2}, \mathtt{message_3}, \mathtt{message_4})$

// *$\mathcal{A}$ returns its guess*

if $b = b'$ **then return** 1

else return 0

FIGURE 10. **The EDHOC PSK identity protection security game. By** `InitRun` **and** `RespRun` **we mean the four algorithms used during the protocol run (two on the initiator side for messages 1 and 3, and two on the responder side for messages 2 and 4).** Endpoints **denotes the sets of all pairs of initiators and responders that can be selected, and** `CRED_I` **and** `CRED_R` **are the standard notations for initiator and responder identities in [10].**

- **Simplified authentication messages.** Messages 2 and 3 use only PSK-based authenitcation, which can break *mutual authentication* and *confidentiality* if:
  (1) A learned PSK allows impersonation. Single-use PSKs exchanged out-of-band limit the impact to one session. However, deriving further PSKs from single-use PSK-based authentication allows continuous impersonation. Multiple-use PSKs exchanged out-of-band allow attacks for the PSK's lifetime. Multiple-use PSKs derived at each session give attackers virtually full impersonation power;
  (2) A learned session secret allows deriving PSKs, recreating the previous scenario.

It should also be noted that PSK1-authenticated EDHOC is vulnerable to *selfie attacks*, which can be avoided by: *(i)* using single-use PSKs or, for multi-use, preventing a device from using the same PSK to authenticate as both initiator and responder; *(ii)* avoiding concurrent sessions of the handshake protocol; *(iii)* not allowing collisions of `G_X` in concurrent ongoing sessions.

- **Modified Key-Schedule.** In PSK1, the key-derivation relies on an ephemeral component (G_XY), which is used to derive salt for a KDF extraction step whose key is the PSK. Authentication keys are derived from both these components, but the encryption key for Message 2 is only derived from the ephemeral component. This weakens external authorization data protection: EAD_1 labels are revealed to passive attackers, and EAD_2 labels are revealed to active attackers who can compute/replay a valid ID_CRED_PSK. This is the same level of protection as in standard EDHOC (methods 0-3).

In short, the following holds for PSK1-authenticated EDHOC:

- The protocol could provide *identity protection*. The protocol does not provide *protection of external data* EAD_1. PSK1 does not resist selfie attacks in *authentication*.
- The protocol provides *confidentiality* and *mutual authentication* as long as PSKs cannot be learned or derived by an adversary.
- PSK1-authenticated EDHOC provides *downgrade protection*, as both parties have to agree on which PSK to use, and the choice of PSK is stored in transcript hash TH_2.

2) PSK2 ANALYSIS

PSK2 has more significant changes for better security/privacy and requires four messages. The protocol differs from EDHOC and PSK1-authenticated EDHOC in the following ways:

- **ID_CRED_PSK is encrypted.** As opposed to PSK1, the identity credentials are sent in Message 3, XOR-encrypted with a keystream derived from the ephemeral secret G_XY. This encryption provides *identity protection* against passive attackers. Active attackers can still MitM to learn it but can't authenticate or learn K_3/K_4-protected content.
- **Simplified authentication messages.** Messages 2 and the first part of message 3 do not use MACs and both parties use PSK-based authentication that occurs through key-confirmation in the second part of Message 3. This conditions *mutual authentication* on the security of the session key, i.e. the protocol's *confidentiality*. The latter holds as long as PSK values are not leaked.
- **Modified Key-Schedule.** As in the case of PSK1, in PSK2-authenticated EDHOC both an ephemeral component and the PSK is used in the key-schedule. The main differences between this variant and both EDHOC and PSK1 are: the absence of MACs, and the fact that PSK is only used in the derivation of PRK_4e3m (and no other previous secret). This makes G_XY insecure against active adversaries (but secure against passive), but AEAD encryption keys in Messages 3/4 are computed using authenticated PRK_4e3m, ensuring external authorization data protection.

In short, the following holds for PSK2-authenticated EDHOC:

- The protocol provides *confidentiality* and *mutual authentication* as long as PSKs cannot be learned or derived by an adversary. Authentication is always explicit. In addition, the protocol provides as much confidentiality as mutual authentication.
- PSK2-authenticated EDHOC provides *downgrade protection* and *external data protection*.
- It provides *identity protection* against passive attackers.

To summarized, we have the following table This can be summarized in the following table:

TABLE 6. Summary of the security properties guaranteed by both protocols, *if the recommendations are followed*. The ✓ symbols indicate that the property is ensured, while the ✗ symbols indicate that the property is only partially ensured or not achieved.

Security property \ Protocol version	PSK1	PSK2
Mutual authentication	✓	✓
Confidentiality	✗	✓
Cryptographic strength	✓	✓
Identity protection	✓	✓
Protection of External Data	✗	✓
Downgrade protection	✓	✓

3) BEST PRACTICES TO MAXIMIZE PSK2 SECURITY

Between PSK1 and PSK2, the latter is the better variant in terms of security and privacy. The protocol can be further improved by enforcing a sound use of PSK material, for instance in the following ways:

- Restrict the number of times that a single PSK can be used. This limits the impact of PKS leakage and of active identity-protection attacks.
- Deriving PSKs as part of the key schedule should only be restricted to session in which authentication was done by either signatures or MACs, that is, by authentication methods 0-3. This limits the impact of PSK leakage.
- An easy and stateless way of preventing selfie attacks is to ensure that PSKs remain single-use.

VII. DISCUSSION

The choice between PSK1 and PSK2 depends on the specific requirements and constraints of the used system. From the results, we see that PSK1 and PSK2 are 1.44× and 1.40× faster than the current Stat-Stat authentication method, respectively. The difference in time between both PSK variants, however, is negligible. As for memory consumption, the Initiator in Stat-Stat method uses 1.22× more flash memory than PSK1 and 1.10× more than PSK2. Regarding the Peak Stack memory usage, all authentication methods show similar results for the Initiator. In the case of the Responder, Stat-Stat consumes 1.13× and 1.06× times more flash memory than PSK1 and PSK2, respectively. The main

difference is observed in the Peak Stack usage in the Responder side, where Stat-Stat uses 1.31× times more stack than PSK1 and 1.17× times more stack than PSK2.

From these results, we can argue that PSK authentication provides performance advantages with respect to Stat-Stat authentication method.

Despite the performance differences between the two proposed PSK variants being negligible, PSK1 introduces an important security risk by sending the credential identifier in the clear in the first message, making the protocol vulnerable against active attackers even in the best case scenario, as well as providing insufficient external data protection for labels `EAD_1` and `EAD_2`. Contrary, PSK2 protects the identity of both the Initiator and the Responder against passive attackers, and provides external data protection unconditionally. Since one of the security requirements of the Working Group is to provide identity protection, PSK2 seems better suited for this purpose.

Although our experiments were limited to the nRF52840 development kit, this platform is a strong representative of the modern resource-constrained IoT device ecosystem due to its widespread use and features, such as the ARM Cortex-M4 processor and cryptographic accelerator. The performance gains observed on this platform suggest that EDHOC PSK is well-suited for a broad range of devices

VIII. FUTURE WORK

Building on these findings, future work could explore several key areas. A formal security analysis of the adopted PSK2 authentication method using tools like Proverif or Tamarin would provide mathematical assurance of its security properties.

Furthermore, it would be valuable to conduct performance evaluations on a more diverse range of hardware platforms, including both lower-end microcontrollers and more powerful gateway devices, to fully understand the protocol's scalability.

Finally, investigating the applicability of EDHOC PSK in other scenarios, such as EAP-EDHOC, and studying its interoperability with established protocols like DTLS-PSK would be crucial steps toward broader adoption.

IX. RELATED WORK

A. EDHOC

EDHOC's development involves security analyses, paralleling TLS 1.3's standardization and formalization. These analyses are symbolic, where cryptographic primitives are black boxes and messages are terms, and computational (more realistic), defining messages as bitstrings and primitives as functions. Lopez-Perez et al. [20] detail the different analyses that have been done. Works by Norrman et al. [21], Cottier and Pointcheval [15] and Ferreira et al. [22] propose the addition of a fourth message in authentication methods involving static Diffie-Hellman keys to guarantee injective agreement on the session key material, enhance security and ensure key confirmation of `PRK_4e3m`, respectively. Subsequent analyses by Kim et al. [23], Jacomme et al. [24] and Günther et al. [14] uncovered privacy and identity vulnerabilities, leading to improvements such as modifying the key derivation process and using full credentials in transcript hashes.

These analyses have collectively contributed to EDHOC meeting its security goals, including confidentiality, mutual authentication and identity protection. Following this approach, we call for formal verification and computational proofs of the PSK authentication mechanism described in our research.

B. USE OF PSK IN OTHER IOT PROTOCOLS

PSK authentication is used in various protocols for constrained environments.

1) TRANSPORT LAYER SECURITY (TLS)

The TLS protocol introduced PSK-based cipher suites in 2005, RFC4279 [25]. TLS 1.3, standardized in RFC8446 in 2018 [26], improved PSK usage by introducing two modes: pure PSK and PSK with Diffie-Hellman Exchange (DHE). PSK with DHE combines PSK with Diffie-Hellman to provide Forward Secrecy. Cremers et al. [27] developed a symbolic model of the latest specification of TLS 1.3 that considers all the possible interactions of the available handshake modes. The paper proved the majority of the specified security requirements of TLS 1.3, including Perfect Forward Secrecy (PFS) , peer authentication, and Key Compromise Impersonation (KCI) resistance. They confirmed that PFS holds in PSK with DHE but not in the pure PSK mode.

2) DATAGRAM TRANSPORT LAYER SECURITY (DTLS)

The DTLS protocol, the datagram variant of TLS, adopted PSK support similar to TLS. DTLS-PSK is detailed in RFC4279, and the recent DTLS 1.3, standardized in RFC9147 [28] in 2022, aligns its PSK usage with TLS 1.3. Restuccia et al. [29] give an overview of DTLS/TLS 1.2 and 1.3 protocol specifications, providing the first experimental comparative performance analysis of various configurations of these protocols on real IoT devices based on low-power microcontrollers. They show that PSK offers a more resource-efficient solution in comparison with ECDHE.

3) 5G AUTHENTICATION AND KEY AGREEMENT (5G-AKA)

In the field of mobile networks, the 3rd Generation Partnership Project (3GPP) has developed specifications for authentication methods different cellular network generations. The 5G Authentication and Key Agreement (5G-AKA) protocol, specified in 3GPP TS 33.501, has become the primary authentication method in 5G networks. It uses a shared secret key stored in the SIM card and the network's authentication center. Basin et al. [30] performed a comprehensive formal analysis of the 5G AKA protocol, showing that some authentication properties are violated prior to key confirmation.

X. CONCLUSION

In this article, we compare two different PSK authentication methods in EDHOC (PSK1 and PSK2) and provide a recommendation to the Working Group to which variant to standardize. To measure performance, both variants were integrated into the *lakers* implementation of EDHOC in Rust. Evaluation is done using the nRF52840 development kit, with both peers communicating over BLE radios. Metrics include number of operations, showing that PSK1 requires 1 less symmetric operation compared to PSK2, memory and energy consumption, with PSK2 requiring $\times$ 1.11 more flash memory and same peak stack usage than PSK1 (for the Initiator), handshake duration, with PSK1 being 2.26 ms faster than PSK2, and security and privacy considerations. We also provide comparison with the current implemented authentication method (Stat-Stat). In spite of PSK1 being slightly more performant, it introduces a security risk sending the credential identifier in the clear in Message 1. Since identity protection is contemplated as a crucial requirement, the Working Group has opted to standardize PSK2 based on our analysis, as it protects both the Initiator's and Responder's identity against passive attackers.

ACKNOWLEDGMENT

The views reflected herein do not necessarily reflect the opinion of French Government nor of European Commission. The authors would like to thank the participants of the IETF LAKE Working Group for their input during the standardization process. This project was funded thorugh the French government under the France 2030 ANR program PEPR Networks of the Future', grant agreement ANR-22-PEFT-0009, the European Union's Horizon Europe Framework Program under Grant Agreement No. 101093046 (project OpenSwarm').

REFERENCES

[1] P. Sun, S. Shen, Y. Wan, Z. Wu, Z. Fang, and X.-Z. Gao, "A survey of IoT privacy security: Architecture, technology, challenges, and trends," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34567–34591, Nov. 2024.

[2] A. Rakshe and N. Dongre, "Survey on security protocols for IoT," in *Proc. IEEE 9th Int. Conf. Converg. Technol. (I2CT)*, Apr. 2024, pp. 1–5.

[3] V. D. Ganda, R. Chhikara, P. S. Mehra, and D. Chawla, "A systematic review on Internet of Things (IoT) security: Applications, architecture, challenges and solutions," in *Proc. 1st Int. Conf. Adv. Comput. Emerg. Technol. (ACET)*, Aug. 2024, pp. 1–8.

[4] T. Watteyne, L. Doherty, J. Simon, and K. Pister, "Technical overview of SmartMesh IP," in *Proc. 7th Int. Conf. Innov. Mobile Internet Services Ubiquitous Comput.*, Taichung, Taiwan, Jul. 2013, pp. 547–551.

[5] M. K. Hasan, Z. Weichen, N. Safie, F. R. A. Ahmed, and T. M. Ghazal, "A survey on key agreement and authentication protocol for Internet of Things application," *IEEE Access*, vol. 12, pp. 61642–61666, 2024.

[6] F. Thabit, O. Can, A. O. Aljahdali, G. H. Al-Gaphari, and H. A. Alkhzaimi, "Cryptography algorithms for enhancing IoT security," *Internet Things*, vol. 22, Jul. 2023, Art. no. 100759. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2542660523000823

[7] E. Lopez-Perez, G. S. Selander, J. Preuß Mattsson, and R. Marin-Lopez, "EDHOC PSK authentication," Internet Engineering Task Force, Tech. Rep. draft-lopez-lake-edhoc-psk-01, Jul. 2024. [Online]. Available: https://datatracker.ietf.org/doc/draft-lopez-lake-edhoc-psk/01/

[8] H. Krawczyk, "SIGMA: The 'SIGn-and-MAc' approach to authenticated Diffie–Hellman and its use in the IKE protocols," in *Proc. Adv. Cryptol.-CRYPTO*, 2003, pp. 400–425.

[9] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," Internet Engineering Task Force, Tech. Rep. RFC9528, May 2010. [Online]. Available: https://www.rfc-editor.org/info/rfc5869

[10] G. Selander, J. P. Mattsson, and F. Palombini, "Ephemeral Diffie–Hellman over COSE (EDHOC)," Internet Engineering Task Force, Tech. Rep. RFC9529, Mar. 2024. [Online]. Available: https://www.rfc-editor.org/info/rfc9528

[11] G. Fedrecheski, M. Vučinić, and T. Watteyne, "Performance comparison of EDHOC and DTLS 1.3 in Internet-of-Things environments," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2024, pp. 1–6.

[12] M. Vučinić, G. Selander, J. P. Mattsson, and D. Garcia-Carillo, "Requirements for a lightweight AKE for OSCORE," Internet Engineering Task Force, Tech. Rep. draft-ietf-lake-reqs-04, Jun. 2020.

[13] H. M. Davis and F. Günther, "Tighter proofs for the SIGMA and TLS 1.3 key exchange protocols," in *Proc. ACNS*, 2021, pp. 448–479.

[14] F. Günther and M. I. T. Mukendi, "Careful with MAc-then-SIGn: A computational analysis of the EDHOC lightweight authenticated key exchange protocol," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy (EuroS&P)*, Jul. 2023, pp. 773–796.

[15] B. Cottier and D. Pointcheval, "Security analysis of improved EDHOC protocol," in *Proc. Int. Symp. Found. Pract. Secur. (FPS)*, 2022, pp. 3–18.

[16] N. Drucker and S. Gueron, "Selfie: Reflections on TLS 1.3 with PSK," *J. Cryptol.*, vol. 34, no. 3, p. 27, May 2021.

[17] C. Brzuska, H. Jacobsen, and D. Stebila, "Safely exporting keys from secure channels: On the security of EAP-TLS and TLS key exporters," in *Proc. Adv. Cryptol.-EUROCRYPT*, vol. 9665, 2016, pp. 670–698.

[18] G. Arfaoui, X. Bultel, P.-A. Fouque, A. Nedelcu, and C. Onete, "The privacy of the TLS 1.3 protocol," in *Proc. Priv. Enhancing Technol.*, 2019, pp. 190–210.

[19] P. Eronen, H. Tschofenig, H. Zhou, and J. A. Salowey, *Transport Layer Security (TLS) Session Resumption Without Server-Side State*, document RFC 5077, Jan. 2008. [Online]. Available: https://www.rfc-editor.org/info/rfc5077

[20] E. L. Pérez, G. Selander, J. P. Mattsson, T. Watteyne, and M. Vučinić, "EDHOC is a new security handshake standard: An overview of security analysis," *Computer*, vol. 57, no. 9, pp. 101–110, Sep. 2024.

[21] K. Norrman, V. Sundararajan, and A. Bruni, "Formal analysis of EDHOC key establishment for constrained IoT devices," 2020, *arXiv:2007.11427*.

[22] L. Ferreira, "Computational security analysis of the full EDHOC protocol," in *Proc. Cryptographers' Track RSA Conf. (CT-RSA)*, May 2024, pp. 25–48.

[23] J. Kim, D. G. Duguma, S. Lee, B. Kim, J. Lim, and I. You, "Scrutinizing the vulnerability of ephemeral Diffie–Hellman over COSE (EDHOC) for IoT environment using formal approaches," *Mobile Inf. Syst.*, vol. 2021, pp. 1–18, Sep. 2021.

[24] C. Jacomme, E. Klein, S. Kremer, and M. Racouchot, "A comprehensive, formal and automated analysis of the EDHOC protocol," in *Proc. USENIX Secur. Symp. (USENIX)*, Aug. 2023, pp. 5881–5898.

[25] H. Tschofenig and P. Eronen, "Pre-shared key ciphersuites for transport layer security (TLS)," Internet Engineering Task Force, Tech. Rep. RFC4279, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/info/rfc4279

[26] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," Internet Engineering Task Force, Tech. Rep. RFC8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/info/rfc8446

[27] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. Van Der Merwe, "A comprehensive symbolic analysis of TLS 1.3," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1773–1788, doi: 10.1145/3133956.3134063.

[28] E. Rescorla, H. Tschofenig, and N. Modadugu, "The datagram transport layer security (DTLS) protocol version 1.3," Internet Engineering Task Force, Tech. Rep. RFC9147, Apr. 2022. [Online]. Available: https://www.rfc-editor.org/info/rfc9147

[29] G. Restuccia, H. Tschofenig, and E. Baccelli, "Low-power IoT communication security: On the performance of DTLS and TLS 1.3," in *Proc. 9th IFIP Int. Conf. Perform. Eval. Model. Wireless Netw. (PEMWN)*, Dec. 2020, pp. 1–6.

[30] D. Basin, J. Dreier, L. Hirschi, S. Radomirovic, R. Sasse, and V. Stettler, "A formal analysis of 5G authentication," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 1383–1396, doi: 10.1145/3243734.3243846.

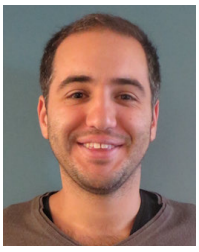
ELSA LOPEZ PEREZ (Student Member, IEEE) was born in Valladolid, Spain, in 1998. She received the B.S. degree in mathematics and physics from the University of Valladolid, in 2021 and the EIT Digital master's degree in data science and artificial intelligence from the University of Saclay and University of Trento, in 2023. She is currently pursuing the Ph.D. degree with Sorbonne University, Inria, Paris. She is part of the AIO Team in Inria, that designs, models and builds networking solutions based on a variety of the Internet-of-Things (IoT) standards. She is grateful for the support of the ANR through the NF-HiSec subproject of the PEPR 5G. She is fluent in English, Spanish, and French.

THOMAS WATTEYNE (Senior Member, IEEE) received the M.Sc. degree in networking, the M.Eng. degree in telecommunications, and the Ph.D. degree in computer science from INSA Lyon, France, in 2005 and 2008, respectively. From 2005 to 2008, he was a Research Engineer with France Telecom, Orange Laboratories. He was a Postdoctoral Research Lead in Prof. Kristofer Pister's Team with the University of California, Berkeley. He is an insatiable enthusiast of low-power wireless technologies. He holds a senior research director position with Inria, Paris, where he leads the AIO Research Team that designs, models and builds networking solutions based on a variety of Internet-of-Things (IoT) standards. He is Wireless System Architect with Analog Devices, the undisputed leader in supplying low power wireless mesh networking solutions for critical applications for industrial and beyond. Since 2023, he has been the Scientific Coordinator of the Horizon Europe OpenSwarm project. He founded and co-leads Berkeley's OpenWSN project, an open-source initiative to promote the use of fully standards-based protocol stacks for the IoT. He is fluent in four languages. In 2019, he co-founded Wattson Elements, the company that develops the award-winning Falco marina management solution. From 2013 to 2022, he co-chaired the IETF 6TiSCH working group to standardize how to use IEEE802.15.4e TSCH in IPv6-enabled mesh networks.

RAFAEL MARIN-LOPEZ is a full-time Assistant Lecturer with the Department Information and Communications Engineering, University of Murcia, Spain. Additionally, he is collaborating actively in IETF, especially in LAKE and EMU Working Groups. He is co-author of RFC 9061, RFC 7499 and other standards. His main research interests include network access authentication, key distribution and security in different type of networks, such as SDN, IoT, and mobile networks.

CRISTINA ONETE is currently an Associate Professor with the University of Limoges, where she co-leads the CRYPTIS Research Team. She is grateful for the support of the ANR through the NF-HiSec subproject of the PEPR 5G and project PRIVA-SIQ.. A Scientific Coordinator of ANR-funded collaborative-research project PRIVA-SIQ and the Chair of Excellence on the Cybersecurity, Privacy and Trust in biomedical systems co-funded by the Limoges Métropole and the Partner Foundation of the University of Limoges, she focuses on the provable security and privacy of cryptographic protocols. Her research includes work on authenticated key-exchange protocols such as TLS, asynchronous messaging such as signal, quantum-resistant protocols, and schemes native to 5G networks and beyond.

CLÉMENT PAPON was born in Clermont-Ferrand, France, in 2001. He received the master's degree in pure and fundamental mathematics, with a specialization in number theory and cryptography, from Université Clermont Auvergne, in 2023. He is currently pursuing the Ph.D. degree with the XLIM Research Laboratory, University of Limoges, Limoges. He is grateful for the support of the ANR through the NF-HiSec subproject of the PEPR 5G. He is a member of the CRYPTIS team, and his work mainly focuses on the provable security of cryptographic protocols involving lawful interception.



MALIŠA VUČINIĆ received the Engineering degree from the University of Montenegro, in 2010, the joint master's degree (Hons.) from the Politecnico di Torino and the Grenoble Institute of Technology, in 2012, and the Ph.D. degree from the Grenoble Alps University, in 2015. He is currently a Research Scientist with the AIO Team of the Inria Research Institute, Paris, France. In parallel with his Ph.D. studies, he was a Research Engineer with STMicroelectronics, and a Visiting Scholar with the University of California, Berkeley, in 2015. He is a Core Developer of the OpenWSN project, the reference 6TiSCH implementation, and a Co-Lead of the 6TiSCH simulator. He led the security standardization work in IETF 6TiSCH, specifying the Constrained Join Protocol (CoJP) for 6TiSCH networks. He is fluent in English, French, Italian, and Montenegrin. He co-chairs the IETF LAKE working group which standardizes a lightweight authenticated key exchange protocol for the Internet-of-Things use cases.